Введение

Актуальность

Современные системы сталкиваются с проблемой обработки и хранения большого объема данных, в таких системах база данных часто оказывается узким местом. Производительность базы данных значительно зависит от ее настройки и для более эффективной ее работы производят подстройку под определенный профиль нагрузки. Подстройка под профиль или, как ее еще называют тюнинг требует глубоких знаний базы, значительных усилий по экспериментам и тестированию, что делает этот процесс долгим, дорогим и недоступным большинству разработчиков и администраторов.

Проблема исследования

Существующие подходы к автоматической настройке БД опираются на эвристики, не учитывающие специфику конкретной нагрузки, не поддерживаются или являются закрытыми решениями.

Цель и задачи

Цель - разработка прототипа системы автоматической оптимизации конфигурации PostgreSQL на основе метрик
Задачи:

1. Провести анализ существующих методов и инструментов автоматической настройки PostgreSQL.
2. Определить ключевые конфигурационные параметры, оказывающие наибольшее влияние на производительность.
3. Разработать архитектуру системы

Объект и предмет исследования

Объект - процесс настройки конфигурации PostgreSQL
Предмет исследования — метод автоматической оптимизации параметров

Методы и инструменты

В работе применяются методы системного анализа, экспериментального моделирования, а также инструменты нагрузочного тестирования. Для поиска

оптимальных конфигураций используется фреймворк Optuna, используемый для гиперпараметрической оптимизации.

Значимость

Научная значимость - использование современных методов оптимизации для настройки СУБД
Практическая значимость - переиспользование результатов работы для создания открытого инструмента для оптимизации или разработки самоподстраивающихся СУБД

Основная часть

Глава 1. Анализ литературы

1.1 Классификация подходов по оптимизации

1. **Ручной** - наиболее распространенный метод. Подразумевает специалиста, который на основе своего опыта подбирает значения
2. **Эвристические** - второй по распространённости метод. Подбор параметров по официальным рекомендациям и соотношениям принятым в сообществе
3. **Алгоритмические** - подбор рекомендации осуществляется с помощью составления функции и ее минимизации
4. **ML** - решения, в основном основанные на обучении с подкреплением. Требует значительных усилий для внедрения и использования.

1.2 Анализ существующих решений

pg_stat_statements

Расширение для postgreSQL для сбора статистики по SQL-запросам, один из самых широко используемых инструментов для ручного анализа.
Для каждого запроса сохраняется:

- количество вызовов
- общее время выполнения
- среднее, минимальное и максимальное время выполнения
- количество прочитанных/записанных блоков
- количество строк, возвращённых или затронутых
- и т.д.

Данными, собираемыми эитм расширением пользуются многие инструменты, облегчающие анализ

PGtune

это утилита для автоматической генерации рекомендаций по настройке параметров конфигурации PostgreSQL на основе характеристик сервера и характера рабочей нагрузки (OLTP, OLAP).
В расчет берутся:

- объём оперативной памяти (RAM)
- количество CPU-ядер
- тип рабочей нагрузки (OLTP, OLAP, смешанная, веб-сервер и т.д.)
- количество одновременных подключений

Параметры подбираются на основе эмпирических правил. Позволяет получить разумную конфигурацию для последующей настройки, дает хорошее начальное значение.

OtterTune

это система автоматической настройки параметров СУБД. Она использует машинное обучение для рекомендации оптимальных значений конфигурационных параметров PostgreSQL (а также MySQL, Oracle и других СУБД) с целью максимизации производительности.
Если верить авторам, то система подбирает параметры не хуже, а, в большинстве случаев, лучше, чем человек
Алгоритм работы

1. **Собирает метрики** с вашей СУБД под реальной или синтетической нагрузкой.
2. **Сравнивает их с историей** успешных/неуспешных настроек
3. **Предсказывает**, какие параметры улучшат целевую метрику
4. **Итеративно предлагает новые конфигурации** и уточняет рекомендации на основе обратной связи.

Состояние проект на данный момент - исходный код открыт и доступен, но проект не поддерживается уже 5-6 лет

Другие решения

- **Azure SQL Database Automatic Tuning**
    - Автоматическое управление индексами
    - Коррекция планов выполнения
- **SageDB** - проект в работе от MIT. Предполагается создание базы данных, каждый компонент которой подстраивается под нагрузку

Пробелы

Как можно заметить, выше перечисленные инструменты не поддерживаются, являются проприетарными или не учитывают специфику конкретной нагрузки.

Глава 2. Методология и план

2.1 Описание предполагаемых методов

Методы оптимизации

Для оптимизации конфигурации предполагается использовать Байесовскую оптимизацию, так как вычисление функции “дорогое”, каждый запуск требует значительного времени и мы не знаем зависимости между параметрами и производиельностью.

Конкретные методы:

- **TPE** - метод поддерживает смешанные параметры, масштабируется до ~50 параметров
- **SMAC** - работает со смешанными типами, условными параметрами, масштабируется до ~100 параметров
- **GridSearch** - не байесовская оптимизация, будет использоваться в качестве контрольного

Критерии

Планируется смоделировать 4 ситуации с соответствующими метриками

1. OLTP нагрузка и пропускная способность
2. OLAP нагрузка и пропускная способность
3. OLTP нагрузка и задержка
4. OLTP нагрузка и взвешенные метрики

2.2 Этапы выполнения

Подготовка тестовой среды и определение рабочих нагрузок

- Развертывание изолированного тестового окружения
- Формирование набора репрезентативных рабочих нагрузок:
    - OLTP-сценарии
    - OLAP-сценарии
- Определение базовых метрик производительности для стандартной конфигурации

Анализ параметров и формирование пространства поиска

- Отбор наиболее значимых параметров
- Определение допустимых значений
- Классификация параметров по типам

Реализация прототипа

- Реализация на Python
- Тестирование

Эксперименты и оценка

- Определение контрольных конфигураций и их оценка
  - Стандартная конфигурация
  - PGTune
- Эксперименты с различными оптимизаторами
  - GridSearch
  - TPE
  - SMAC

2.3 Инструменты

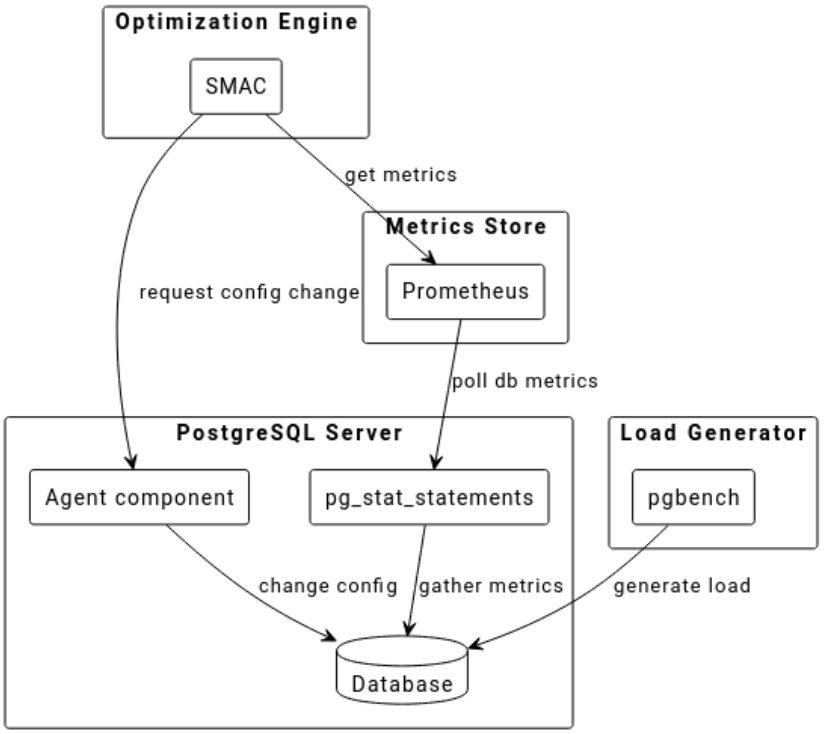
- **СУБД:** PostgreSQL 18
- **Язык программирования:** Python
- **Фреймворки и библиотеки:**
  - **Optuna** - фреймворк для гиперпараметрической оптимизации с нужными мне оптимизаторами
- **Инструменты нагрузочного тестирования:**
  - pgbench (встроенный в PostgreSQL)
  - sysbench для генерации смешанных нагрузок
  - BenchBase - инструмент, поддерживающий TPC бенчмарки
- **Машина:**
  - Личный ноутбук
  - Тестовый сервер
- **Другие**
  - git
  - docker - для изоляции

2.4 Ожидаемые результаты

Ожидаю улучшение на 10-15% по сравнению с контрольными конфигурациями для каждого вида нагрузки и метрики

Глава 3. Архитектура и реализация

Архитектура



Спроектированная архитектура приложения предполагает несколько компонентов:

1. Агент - скрипт, находящийся на той же машине, что и база данных. Предназначен для изменения файла конфигурации, перезагрузки, ожидания запуска СУБД
2. Prometheus хранилище и сборщик метрик
3. Оптимизатор - программа, содержащая основную логику оптимизации, предлагает новую конфигурацию, читает метрики из Prometheus. Алгоритм, используемый для оптимизации - SMAC
4. Генератор нагрузки
5. Непосредственно СУБД

Реализация

Прототип реализован с использованием:

- Python
- Optuna
- SMAC

Работа алгоритма идет по следующему циклу:

1. SMAC предлагает новый набор параметров
2. Агент применяет конфигурацию СУБД.
3. Запускается 5-минутный прогон pgbench.
4. Prometheus фиксирует средний TPS и возвращает значение оптимизатору.
5. Алгоритм обновляет мета-модель и переходит к следующей итерации.

Глава 4. Эксперименты

Методика

Генератор нагрузки	pgbench (TPC-B like профиль)
Длительность прогона	5 минут
Целевая метрика	TPS (Transactions Per Second)
Базовая конфигурация	Рекомендации pgTune
Параметры поиска	18 параметров
Количество итераций	60

Эксперименты проводились на локальной машине с использованием docker и с ограничением ресурсов его средствами

Результаты

В ходе экспериментов получено следующее:

- После 60 итераций достигнут прирост производительности: +10% по TPS относительно конфигурации pgTune.
- При этом основная часть улучшения достигается за первые 15–20 итераций, далее идёт тонкая настройка.

Результат подтверждает гипотезу: алгоритм SMAC способен находить конфигурации, превосходящие эвристические рекомендации, без участия эксперта.

Заключение

В ходе НИР обоснована актуальность проблемы оптимизации конфигурации PostrgeSQL и проанализированы недостатки существующих решений
Сформулирована цель - разработать прототипа системы автоматической оптимизации конфигурации PostgreSQL на основе метрик с использованием фреймворка Optuna и методов байесовской оптимизации (TPE, SMAC).
Составлен поэтапный план реализации проекта от подготовки тестовой среды до проведения и анализа экспериментов.
Прототип реализован и протестирован.
Проведены первые эксперименты, в ходе которых получилось достигнуть прирост производительности в 10%.
Получен прототип open-source решения, способный повысить производительность PostgreSQL на 10%, который в будущем ляжет в основу более зрелых решений.

Список использованных источников

1. Б.А. Новиков Г.Р. Домбровская. Книга Настройка приложений баз данных. — БХВ-Петербург, 2006.
2. Optuna: A hyperparameter optimization framework [Электронный ресурс]. — Режим доступа: https://optuna.org, свободный. — Загл. с экрана. — Яз. англ. — Дата обращения: 21.01.2026.
3. Bergstra, J. Algorithms for Hyper-Parameter Optimization / J. Bergstra, R. Bardenet, Y. Bengio, B. Kégl // Advances in Neural Information Processing Systems 24 (NIPS 2011). — 2011. — P. 2546–2554. — URL: https://proceedings.neurips.cc/paper/2011/file/86e8f7ab32cfd12577bc2619bc635690-Paper.pdff (дата обращения: 21.01.2026).
4. Lindauer, M. SMAC3: A Versatile Bayesian Optimization Package for Hyperparameter Optimization / M. Lindauer, K. Eggensperger, M. Feurer [et al.] // Journal of Machine Learning Research. — 2022. — Vol. 23, № 54. — P. 1–9. — URL: https://doi.org/10.48550/arXiv.2109.09831(дата обращения: 21.01.2026).
5. Van Aken, D. Automatic Database Management System Tuning Through Machine Learning / D. Van Aken, A. Pavlo, G. J. Gordon, B. Zhang // Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD '17). — Chicago, IL, USA, 2017. — P. 1235–1248. — DOI: 10.1145/3035918.3064029. — URL:

https://www.cs.cmu.edu/~ggordon/van-aken-etal-parameters.pdf (дата обращения: 21.01.2026).
6. Liu, Y. Automatic Configuration Tuning on Cloud Database: A Survey / Y. Liu, Z. Chen, X. Wang [et al.] // arXiv preprint arXiv:2404.06043. — 2024. — URL: https://arxiv.org/pdf/2404.06043 (дата обращения: 21.01.2026).
7. Cui, Y. Facilitating Database Tuning with Hyper-Parameter Optimization: A Comprehensive Experimental Evaluation / Y. Cui, H. Zhang, J. Li [et al.] // Proceedings of the VLDB Endowment. — 2022. — Vol. 15, № 9. — P. 1808–1821. — DOI: 10.48550/arXiv.2110.12654. — URL: https://www.vldb.org/pvldb/vol15/p1808-cui.pdf (дата обращения: 21.01.2026).
8. SageDB: A Self-Assembling Database System [Электронный ресурс] / MIT DSAIL. — URL: https://dsail.csail.mit.edu/index.php/projects/ (дата обращения: 21.01.2026).